

05. REPRODUCIBLE MACHINE LEARNING PIPELINE FOR CONSTRUCTION COST AND SCHEDULE PREDICTION

Doctoral Candidate: Jhon Wilbert Quiroz Quilla

Course: Research Methods and Scientific Integrity in AI and Advanced Technologies

Program: Doctoral Program in Deep Technologies – UNMSM

1. INTRODUCTION: WHY REPRODUCIBILITY MATTERS

When I started working in construction project management, I quickly realized that one of our biggest problems was not a lack of data, but a lack of trust in data. We had records of past projects—technical files, metrados, valuations, monthly reports, liquidations—but they were scattered across different computers, different formats, and different people's memories. When a project manager left the company, their knowledge left with them. When we started a new project, we often repeated the same mistakes because we could not remember what we had learned from the previous one.

This is the problem that reproducibility in machine learning seeks to solve. A reproducible ML pipeline is not just about making code run on someone else's computer; it is about building institutional memory. It is about saying: "This is how we did it, these were the data we used, these were the assumptions we made, and you can check our work." It is about making our knowledge explicit, transparent, and transferable.

In this section, I present the reproducible ML pipeline that I am building for my Digital Twin model. This pipeline is designed to be the technical heart of my research—the part that ensures that when I say my model predicts cost overruns with a certain accuracy, anyone else can verify that claim.

2. THE REPRODUCIBILITY PYRAMID

In class, Dr. Loveleen emphasized that reproducibility requires multiple layers working together. I think of this as a pyramid, where each layer supports the ones above it. If any layer is missing, the whole structure is unstable.

What I Learned in Class

During one of our sessions, Dr. Loveleen shared a shocking statistic: researchers at Princeton tried to reproduce 255 machine learning papers and could fully reproduce only about 14%. The main culprits were not complicated algorithms or missing code, but simple things: no one tracked which version of the data was used, no one recorded the random seed, no one documented the environment. These are mistakes that are completely avoidable, but only if we are intentional about reproducibility from the start.

This is why I am building my pipeline the way I am—not because it is the easiest way, but because it is the only way that allows my work to be verified, trusted, and built upon by others.

The Layers

Layer 1: Code Version Control (Git + GitHub) — Every line of code I write is tracked. If something goes wrong, I can go back to any previous version. When I share my repository, anyone can see exactly what code was used.

Layer 2: Data Version Control (DVC) — My data is not stored in Git (it is too large), but DVC tracks every version of every dataset. When someone clones my repository and runs the DVC pull command, they get exactly the same data I used.

Layer 3: Experiment Tracking (MLflow) — Every time I run an experiment, MLflow records the parameters, metrics, and artifacts. I can compare different runs and see what changed.

Layer 4: Environment Control (Docker) — My code runs inside a container that has all the dependencies it needs. No more "it works on my machine."

Layer 5: Compute (Google Colab) — I use Colab because it is free and accessible. I record the GPU type and any other relevant hardware details so that others know the conditions under which my results were obtained.

3. MY PROJECT STRUCTURE

I have organized my repository following the structure recommended in the Project Brief. This is not arbitrary—each folder corresponds to a specific deliverable, and the structure makes it easy for anyone to navigate the project.

The Repository Layout

The root of my repository contains the README file, which explains the project overview and provides exact run instructions. It also contains the Dockerfile for environment definition, the requirements.txt file with pinned dependencies, the DVC configuration folder, and the DVC pipeline definition file.

The data folder is organized into three subfolders: raw for the original data from technical files and project documents, processed for cleaned and structured data, and external for any reference data that might be needed. The notebooks folder contains Jupyter or Colab notebooks for exploratory analysis and model development.

The source code is organized in the src folder, with separate files for data processing, feature engineering, model definitions, training, evaluation, and pipeline orchestration. The models folder stores saved models, and the results folder contains metrics and outputs. The mlruns folder is used for MLflow tracking. Additional documentation includes a Model Card, a Dataset Datasheet, and a reflective writing log.

Why This Structure

I chose this structure because it separates concerns. The data is separate from the code, which is separate from the models, which is separate from the results. This makes it

easier to track what changed and to reproduce any part of the pipeline independently. It also follows the principles of the project brief, which I have found helpful as a guide.

4. THE ML PIPELINE: STEPS AND STAGES

Stage 1: Data Loading and Splitting

My data comes from the construction projects I have worked on—technical files, metrados, valuations, monthly reports, and liquidations. These are real project records, not synthetic data. However, they are not in a format ready for machine learning. The first step is to load them and convert them into a structured format.

I start by loading all the project data into a structured format using Python's pandas library. The data is then split before any preprocessing occurs. I divide the data into 70% for training and 30% for testing. The test set is kept completely separate and is only used once at the very end to evaluate the final model.

Why this matters: I learned in class that one of the most common mistakes in machine learning is data leakage—when information from the test set accidentally influences the training process. To prevent this, I split the data before doing anything else. I also set the random seed to 42 to ensure that the split is reproducible. The test set is kept completely separate and is only used once, at the very end, to evaluate the final model.

Stage 2: Feature Engineering

The raw data from construction projects is messy. Different projects use different formats. Some have missing values. Some have outliers. Before I can train a model, I need to clean and transform the data.

The first step in this stage is handling missing values. I use median imputation for numerical variables, which means I replace missing values with the median of that variable. For categorical variables, I use mode imputation or create a separate "unknown" category.

The second step is standardization. I scale the numerical features so that they have a mean of zero and a standard deviation of one. This is important because many machine learning algorithms are sensitive to the scale of the input features.

The third step is creating derived features. This is where domain knowledge matters most. For example, I create a feature called "change order density," which is the number of change orders divided by the project duration. This captures the rate of change rather than just the total number of changes. Similarly, I create a "complexity ratio" by dividing the number of floors by the total built area. These derived features often capture patterns that raw features miss.

The challenge: The most important features in construction are often not the ones that come directly from the data. Feature engineering is where domain knowledge matters most, and it is also where I can most easily make mistakes by creating features that are correlated with the outcome in ways that do not generalize.

Stage 3: Model Training

I train multiple models and compare their performance. This is where the "cognitive learning" part of my research comes in—I am not just using one algorithm, but comparing several to see which performs best for construction data.

The models I compare include:

Linear Regression: This is my baseline model. It is simple and interpretable. If the more complex models cannot beat it, then they are not worth the extra complexity.

Random Forest: This is an ensemble of decision trees. It is robust to outliers and captures non-linear relationships well. It also provides feature importance scores, which helps with interpretability.

XGBoost: This is a gradient boosting algorithm that is known for high performance. It often wins machine learning competitions because it is very good at learning complex patterns.

CatBoost: This is similar to XGBoost but handles categorical features natively. Since many construction variables are categorical (project type, location, soil type), this algorithm may perform particularly well.

LSTM Networks: This is a type of neural network designed for sequential data. I am exploring it because construction projects unfold over time, and the cost and schedule outcomes depend on the sequence of events.

For each model, I train it on the training data and evaluate it on the validation data. I compare their performance using metrics like Mean Absolute Percentage Error (MAPE) and R-squared.

Why multiple models: No single algorithm is best for every problem. By comparing multiple models, I can find the one that best fits the patterns in construction data. Based on the literature, I expect XGBoost or CatBoost to perform well.

Stage 4: Hyperparameter Tuning

The default parameters for machine learning models are rarely optimal. I tune the parameters of the best-performing model to squeeze out additional performance.

I use a technique called grid search with cross-validation. This involves defining a grid of possible parameter values and trying every combination. For example, for XGBoost, I might try different values for the number of trees, the maximum depth of each tree, and the learning rate.

The best combination of parameters is chosen based on performance on the validation data. This process is computationally expensive but helps ensure the model performs well.

A confession: This is where I have to be honest with myself. I could keep tuning parameters forever, and each time I might get a slightly better result. But if I keep tuning until I get the result I want, I am no longer doing honest science. The literature review in class reminded me that this is a common form of p-hacking. My commitment is to pre-define the search space and evaluate the model only once on the test set.

Stage 5: Evaluation on Test Set

After all the tuning is done, I evaluate the final model on the test set that has been held aside from the beginning.

This is the only honest evaluation. Everything before that—training, validation, tuning—is part of the learning process. The test set tells me how well my model will perform on new, unseen data. This is what matters in practice.

I report the test set performance using the same metrics I used for validation. I also report the confidence intervals for these metrics, which gives a sense of the uncertainty in the estimates.

5. THE REPRODUCIBILITY TOOLS

5.1 Git: Tracking Every Line of Code

Git is the foundation of reproducibility. Every change I make to my code is committed with a message explaining what I changed and why. This means that if I accidentally break something, I can go back to any previous version.

When I share my repository on GitHub, anyone can see not just the final code, but the entire history of how it was developed. This is transparency.

5.2 DVC: Tracking Data Versions

DVC (Data Version Control) is like Git for data. It stores pointers to data files in Git, while the actual data files are stored in a remote location. For me, this is Google Drive.

This approach offers three key advantages: my Git repository stays small because only code and small pointer files are stored; I can track different versions of my data; and anyone with access to my repository can download exactly the same data I used.

To set this up, I initialize DVC in my repository, add the data directory to DVC, configure a remote storage location, and push the data to that remote. When someone clones my repository, they run the DVC pull command and get exactly the data I used.

5.3 MLflow: Tracking Experiments

MLflow tracks every experiment I run. For each run, it records the parameters I used, the metrics I obtained, and the artifacts I produced, such as models and plots.

This means that when I compare two runs, I know exactly what was different between them. I can say with confidence: "On March 15, with these parameters, I got this result."

To use this, I start a new MLflow run before each experiment, log all the parameters and metrics, and save any artifacts. Later, I can view all my experiments in the MLflow user interface and compare them side by side.

5.4 Docker: Controlling the Environment

Docker is the most powerful tool for environment control. It creates a container that has exactly the dependencies my code needs—the right version of Python, the right libraries, and the right system packages.

I use a lightweight Python 3.11 slim image because it is small and fast to build. I do not need GPU support in Docker because I do my training on Google Colab, and the model evaluation can run on CPU.

The Dockerfile I use starts with the base image, installs any system dependencies like Git and curl, sets up the working directory, copies the requirements file, installs the Python dependencies, and finally copies the rest of the project. Anyone can build this Docker image and run the code in a controlled environment.

5.5 Google Colab: Accessible Compute

I use Google Colab for training because it is free and accessible. This means that anyone with an internet connection can reproduce my experiments without needing to buy expensive hardware.

I record the GPU type and any other relevant details in my results. This is important because floating-point computations can vary across different hardware.

6. THE STRANGER TEST

Dr. Loveleen emphasized in class that a project is only truly reproducible if a "stranger"—someone who has never seen your work before—can clone your repository and reproduce your results.

How I Prepare for the Stranger Test

Step 1: Clear README

My README.md file tells the stranger exactly what to do: clone the repository, install Docker, build the Docker image, run the container, inside the container run the DVC pull command, and then run the pipeline script. The results will be in the results directory. The expected output is also documented, so the stranger knows what to look for.

Step 2: Pinned Dependencies

All dependencies are pinned to specific versions in the requirements.txt file. This prevents the "it worked last month but now it's broken" problem.

Step 3: Fixed Seeds

Every source of randomness is fixed with a specific random seed. This ensures that my results are deterministic.

Step 4: Self-Contained Data

All data is tracked with DVC. The stranger does not need to find data from anywhere else.

Step 5: Full Pipeline

The entire pipeline can be run with a single command. No manual steps, no "you need to run this notebook first, then this script..."

7. MY DATA: A CONSTRUCTION PROJECT MANAGER'S PERSPECTIVE

I want to be honest about the data I am using. As someone who has worked on construction projects for over a decade, I have access to real project data that most researchers do not have. This is both an advantage and a responsibility.

What I Have Access To

Through my work, I have access to the following documents from real construction projects.

Technical files: The original project plans, specifications, and budgets. These tell me what was planned.

Metrados: The quantity takeoffs that translate plans into measurable quantities of work.

Valuations: Monthly assessments of work completed, showing the progress against the plan.

Monthly reports: Detailed accounts of what happened each month—delays, changes, issues.

Liquidations: Final accounts that show the true cost of the project.

Change orders: Records of scope changes and their impact on cost and schedule.

These are real documents from real projects. They are messy, inconsistent, and full of the kind of noise that makes machine learning challenging. But they are also rich with information that could help us predict and prevent future problems.

How I Handle the Messiness

I have learned that in construction, perfection is the enemy of progress. Data is never clean. Projects never follow the plan exactly. Every project has its own unique context. My approach is not to pretend that this messiness does not exist, but to embrace it and build models that can handle it.

I use median imputation for missing values because it is robust to outliers. I use outlier detection to identify projects that are truly exceptional and decide whether to include them or treat them separately. I create domain-specific features to capture the most important patterns. And I compare multiple models to find the approach that best handles the noise.

8. REFLECTION: WHAT SURPRISED ME

When I started building this pipeline, I thought the hardest part would be the machine learning. I was wrong. The hardest part was getting the data into a consistent format.

Different projects use different formats for their technical files. Some use Excel, some use specialized software. Some have complete records, others have gaps. Getting all of this into a single, structured dataset took much longer than I expected.

I also underestimated how much I would learn from the process. When I started, I thought of reproducibility as a technical requirement—something I had to do to satisfy the course requirements. Now I see it differently. Reproducibility is not just about checking boxes. It is about making my thinking explicit and transparent. It is about building something that others can trust and build upon.

There is still work to be done. My pipeline is not yet complete. I need to add more features, test more algorithms, and validate on more projects. But the foundation is in place. I have built something that can be tested, verified, and improved.

9. CONCLUSION

This pipeline is the technical heart of my research. It is how I ensure that my results are reproducible, transparent, and trustworthy. It is how I contribute to building institutional memory in the construction sector.

I have learned that reproducibility is not just about code and tools. It is about a mindset. It is about being willing to share not just your successes, but also your failures. It is about being honest about what you did and what you found.

I have also learned that building a reproducible pipeline is not easy. It takes time and effort. But it is worth it. When I finish this research, I will have something that others can verify, learn from, and build upon. That is what science is supposed to be.

REFERENCES

Data Version Control. (2026). Get Started: Data Pipelines.

MLflow. (2026). MLflow Tracking: A Quick Start Guide.

Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché-Buc, F., Fox, E., & Larochelle, H. (2021). Improving reproducibility in machine learning research: A report from the NeurIPS 2019 Reproducibility Program. *Journal of Machine Learning Research*, 22(164), 1-20.

Sugimura, P., & Hartl, F. (2018). Building a Reproducible Machine Learning Pipeline. *arXiv preprint*.